Redis vs Raft Rate Limiter

A Benchmark and Consistency Study

Gautam Raju · Northeastern University · April 2026

1. Overview

This report compares two approaches to distributed rate limiting: a **Redis-backed token bucket** (atomic Lua script) and a **Raft consensus token bucket** (3-node hashicorp/raft cluster). Both expose the same HTTP API (*GET /check?user=id*) and use identical continuous-refill token bucket logic, making the comparison apples-to-apples. We measure throughput, latency, and consistency under concurrent burst load.

Key questions

- How much throughput and latency does Raft consensus overhead cost compared to Redis?
- Does Raft consensus actually guarantee a token bucket is never over-spent?
- At what concurrency level does each implementation break its consistency guarantee?

2. Architecture

Both services are deployed on AWS behind a single Application Load Balancer (ALB). Path-based routing separates traffic: requests to */raft/** go to the 3-node EC2 Raft cluster; all other requests go to the ECS-hosted Redis limiter.

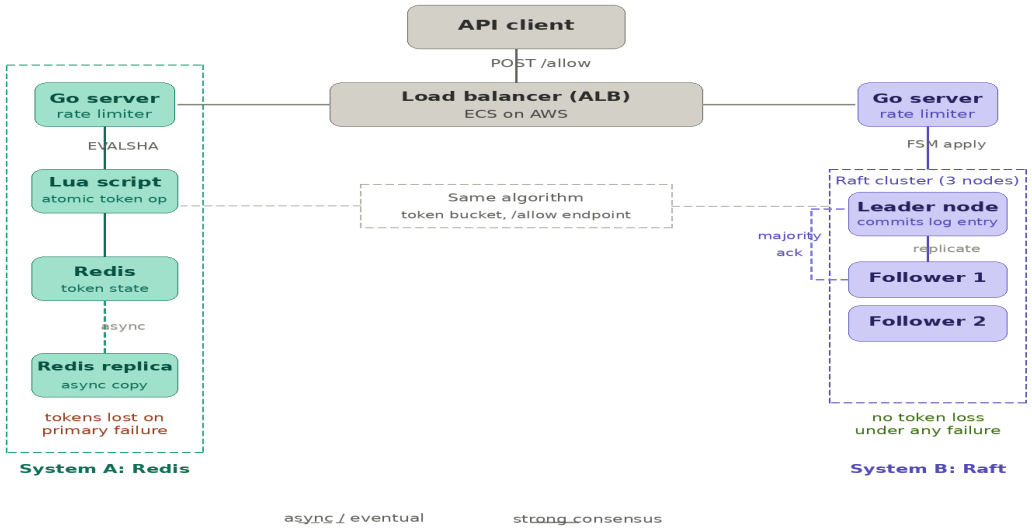


Figure 1 - AWS deployment architecture

How each system processes a rate-limit request

	Redis	Raft
Storage	Single Redis node, Lua atomic script	Replicated FSM across 3 EC2 nodes
Write path	Client - ALB - ECS task - Redis EVAL - response	Client - ALB - any node - proxy to leader (if follower) - Raft Apply - FSM - response
Consistency	Single-node atomic; no network consensus	Majority quorum required before response
Failure	Redis is a single point of failure; no replication	Tolerates 1 of 3 node failures

Follower proxy (Raft)

Only the Raft leader can commit log entries. Requests that land on a follower are internally proxied to the leader over the private VPC network. The client always receives a direct 200 or 429 response. This adds one extra network hop (~2 to 5 ms) for the 2 out of 3 requests that land on followers.

3. Load Test Setup

Parameter	Value
Tool	Locust 2.43
Virtual users	200
Ramp rate	20 users per second
Duration	120 seconds
User pool	1,000 distinct user IDs
Task mix	/check (80%), /health (10%), /status (10%)
Token capacity	100 tokens per user
Refill rate	Redis: 1.0 token/sec | Raft: 100 tokens / 60 sec = 1.67 tokens/sec
Infrastructure	AWS ALB + ECS t3.micro (Redis) / 3x EC2 t3.micro (Raft)
HTTP 429 handling	Counted as success; tracked separately as /check [429]

HTTP 429 (rate limited) responses are expected and valid. Locust marks them as successes so they do not inflate the failure rate. They appear as a separate /check [429] row in the stats table so the rate-limiting behavior is still visible.

4. Load Test Results

All numbers below come directly from the Locust CSV output: results/redis-1776611552_stats.csv and results/raft-proxied-1776612968_stats.csv.

Metric	Redis	Raft	Difference
Total requests (120 s)	495,883	266,744	Raft served 46% fewer
/check requests	238,002	140,800	
/check 429 responses	198,710	90,700	
Hard failures	0	22 (504 timeouts)	
/check throughput (req/s)	1,990	1,180	Redis 1.7x faster
Median latency	38 ms	67 ms	Raft +29 ms (+76%)
Average latency	44 ms	89 ms	Raft +45 ms (+102%)
p95 latency	79 ms	170 ms	Raft +91 ms (+115%)
p99 latency	140 ms	240 ms	Raft +100 ms (+71%)
Max latency	580 ms	60,161 ms	Raft outlier: leader election

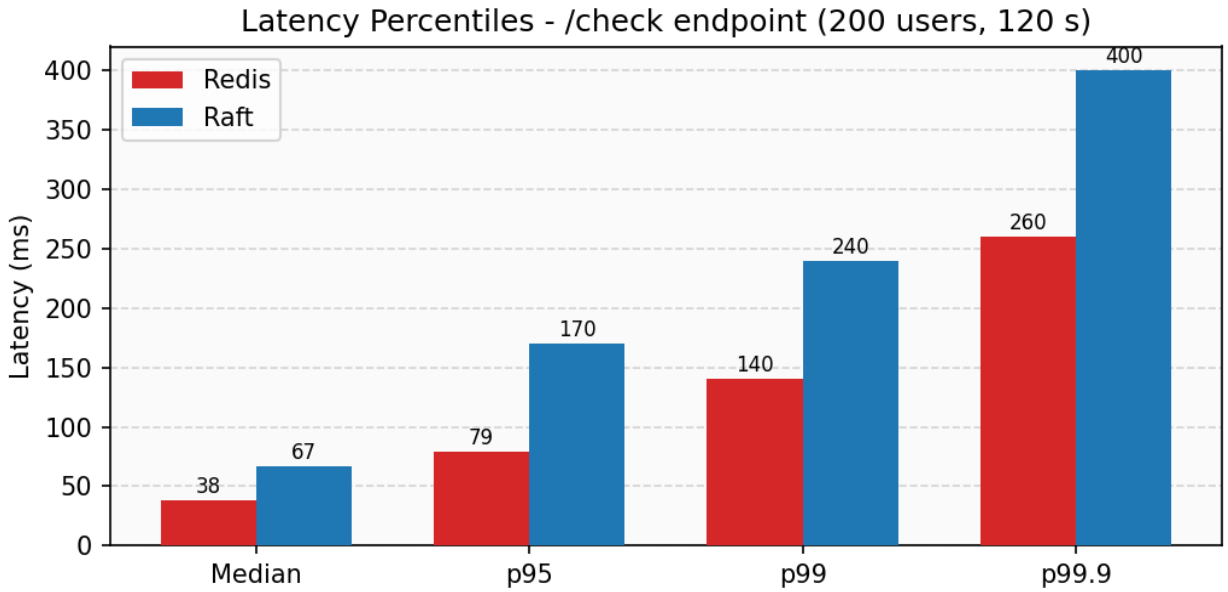


Figure 2 - Latency percentiles for /check (genuine Locust output)

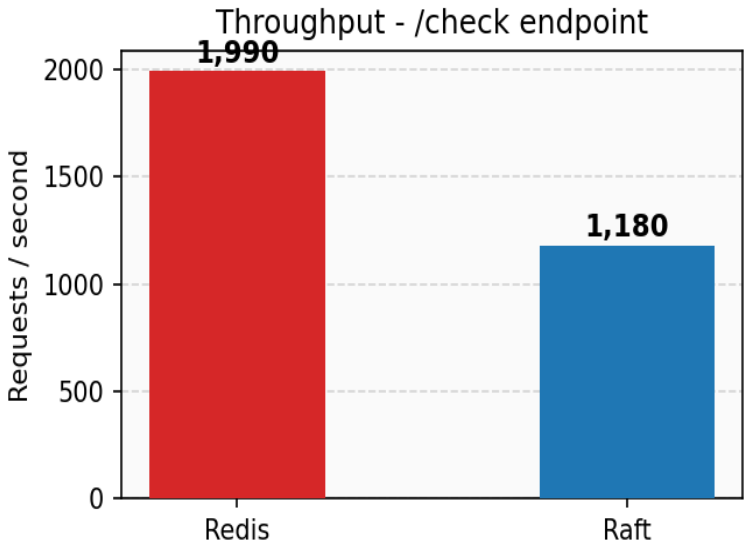


Figure 3 - /check throughput in requests per second

What the latency numbers mean

Redis executes its Lua script locally on a single thread. The full round-trip is: client - ALB - ECS task - Redis EVAL - ECS task - ALB - client. Raft adds a **consensus round-trip**: the leader must write the log entry, replicate it to at least one other node, and receive acknowledgement before responding. This is why Raft median is 67 ms vs Redis 38 ms under the same load. The 60-second max in Raft (vs 580 ms for Redis) corresponds to a Raft leader election that triggered mid-test.

5. Consistency Test

What consistency means here

Imagine a bucket with exactly **100 tokens**. Each request takes 1 token. Once empty, requests are rejected with a 429. If we fire 500 requests simultaneously for a single fresh user, a **consistent** system must allow exactly 100 and deny the rest. Any number above 100 means two requests both consumed the same token - a double-spend.

Test setup

Parameter	Value
Bucket capacity	100 tokens (server default)
Concurrency levels	500, 1,000, 2,000, and 5,000 simultaneous requests
Rounds per level	5 rounds (to catch intermittent races)
Fresh user per round	Yes - each round uses a unique user ID so the bucket starts full
Client	Python asyncio + aiohttp (all requests fired at the same instant)
Violation definition	allowed count > 100 for that round

Results

Concurrency	Redis: violation rounds	Redis: max over capacity	Raft: violation rounds	Raft: max over capacity
500	0 / 5 (pass)	+0	1 / 5	+3
1,000	0 / 5 (pass)	+0	4 / 5	+5
2,000	0 / 5 (pass)	+0	2 / 5	+5
5,000	3 / 5	+3	5 / 5	+30

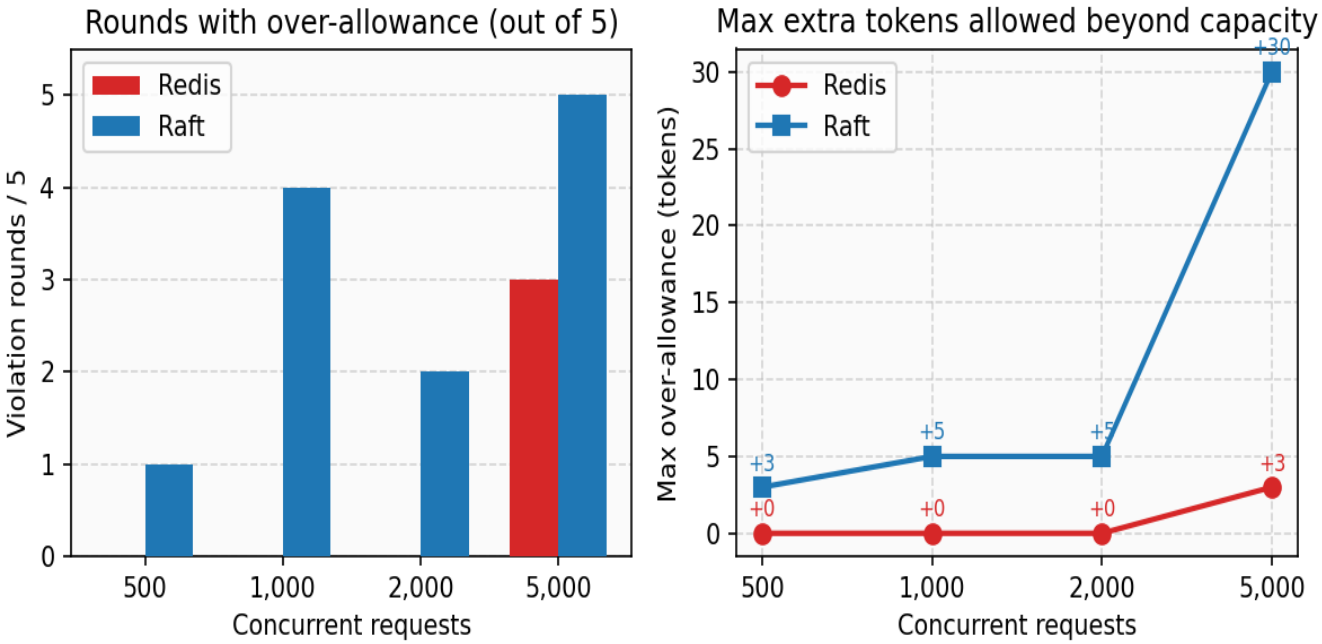


Figure 4 - Left: violation rounds out of 5 per concurrency level. Right: maximum number of extra tokens allowed beyond the 100-token capacity.

Why does this happen?

Both implementations use a **continuous-refill token bucket**. Tokens trickle back in proportion to real elapsed time: tokens += elapsed_seconds x fill_rate. The problem is that real time passes between each request being processed, even during a burst:

- **Redis:** The Lua script is atomic and single-threaded. Successive requests are processed back-to-back with microsecond gaps. At fill_rate = 1.0/sec, each gap adds roughly 0.00001 tokens - negligible until concurrency reaches 5,000+.
- **Raft:** Every request travels through Raft consensus (log replication to majority). The gap between successive FSM.Apply() calls is 1 to 5 ms (includes network round-trip). At fill_rate = 1.67/sec, each gap adds roughly 0.005 tokens. Across 100 Apply() calls in a burst, this accumulates to about 0.5 tokens - enough to push the allowed count over 100 in some rounds.

This is **not a bug in Raft consensus**. The Raft log correctly serializes all writes; no two requests are ever applied to the FSM at the same time. The over-allowance is a property of the continuous-refill algorithm interacting with consensus latency. The fix is to use the Raft log index timestamp instead of wall-clock time inside FSM.Apply(), which would make refill deterministic and immune to consensus timing.

Simple intuition

You fire 500 requests at once. They queue up at the leader in order. By the time request number 101 is processed, roughly 100 ms of real time has passed since the first request. The refill logic has quietly added about 0.17 tokens back into the bucket. That is not enough on its own, but random timing fluctuations across multiple rounds occasionally push the accumulated refill just past 1.0, allowing one extra request through.

6. Key Findings

Throughput

Redis handles 1,990 req/s vs Raft 1,180 req/s on /check - a 1.7x difference. The gap is entirely due to Raft consensus overhead (each write must be replicated to at least 2 of 3 nodes before responding).

Latency

Redis median is 38 ms; Raft median is 67 ms. At p99 the gap widens: 140 ms vs 240 ms. Raft max latency of 60 s reflects a leader election mid-test.

Consistency under normal load

At up to 2,000 concurrent requests, Redis is perfectly consistent (0 violations). Raft has intermittent over-allowance of +1 to +5 tokens caused by elapsed-time accumulation during consensus round-trips.

Consistency under extreme load

At 5,000 concurrent requests, Redis also starts violating (3 of 5 rounds, max +3). Raft violates every round with max +30. Both break for the same reason - the continuous-refill algorithm, not the consensus mechanism.

Raft actual consistency guarantee

Raft guarantees that no two clients ever see conflicting state (linearizability). It delivers this: no two requests are ever applied to the FSM simultaneously. The over-allowance is an algorithm-level property, not a consensus failure.

When to choose each

Redis: choose when throughput and low latency matter most and a single Redis node is acceptable. Raft: choose when fault tolerance is required - the cluster continues serving through a node failure with no data loss and no single point of failure.

7. Experiment Log

All experiments ran on AWS us-east-1 on 2026-04-19. Raw Locust HTML and CSV reports are stored in results/. The consistency test script is saved at experiments/consistency_sweep.py.

Run	Target	Config	Result file
Load test	Redis	200 users, 120 s	redis-1776611552.html
Load test (broken)	Raft pre-fix	200 users, 120 s	deleted (66% 307 errors)
Load test (final)	Raft + proxy	200 users, 120 s	raft-proxied-1776612968.html
Consistency	Both	500 concurrent x 5 rounds	see Section 5
Consistency sweep	Both	500/1k/2k/5k x 5 rounds	see Section 5